"Automate. Integrate. Deliver."

MODULE 44

Continuous Delivery Pipeline

Deliver value to your users faster, safer, and more reliably -- automate the path from code commit to production release and govern it with confidence.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 44 OVERVIEW

Design, implement, and govern a Continuous Delivery pipeline that delivers high-quality software to production reliably, safely, and efficiently.

Building software is only half the challenge -- releasing it safely, predictably, and with the ability to recover fast is equally important.

DURATION & LAB

3 Hours Theory

CD concepts, release management, environments, deployment strategies, rollback, DR, governance, and automation.

90 Minutes Lab

Lab 44: Northstar CRM release path -- immutable artifact promotion, gates, staging rehearsal, and rollback.

Scenario

Northstar CRM Customer Management Platform -- one immutable artifact promoted test to staging to production.

MODULE ARC

Understand CD & Release Mgmt

Why release management matters, the CD lifecycle, and environment strategy from Dev to Production.

Master Deployment & Rollback

Blue-Green, Canary, Rolling, and Recreate strategies, plus rollback, disaster recovery, and change governance.

Build the Lab

Promote an immutable release through objective gates with rehearsed rollback for the Northstar CRM release path.

KEY TAKEAWAY Total time: 3 hours theory + 90 minutes lab. By the end, you will design, promote, and govern a Continuous Delivery pipeline that ships safely and rolls back with confidence.

WHY RELEASE MANAGEMENT MATTERS

Great software is only valuable when it reaches users reliably, safely, and on time. Release management ensures every change delivers business value with confidence.

Without Effective Release Management	Why Release Management Matters
Higher risk of failures and outages.	Reduces risk with structured releases and rollback plans.
Delayed delivery of new features.	Faster time to value via streamlined releases.
Inconsistent deployments across environments.	Ensures consistency with standardized processes.
Unhappy customers and lost trust.	Enhances customer trust through reliable releases.
Higher cost of fixes and rollbacks.	Improves quality via testing and clear release criteria.

KEY OUTCOMES

Reduces Risk **Improves Quality** **Ensures Consistency** **Enhances Customer Trust** **Accelerates Delivery**

Supports Business Goals

KEY TAKEAWAY Effective release management turns code into value -- safely, predictably, and continuously. Plan well. Test thoroughly. Release safely. Improve continuously.

Why Release Management Matters

Deliver the right software, at the right time, for the right impact

BUSINESS OUTCOMES

- Reliable Releases
- Faster Delivery
- Higher Quality
- Lower Costs
- Happier Customers
- Better Business Results

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to design, implement, and govern a Continuous Delivery pipeline that delivers software reliably, safely, and efficiently.

- **1. Understand Core Concepts —** explain the principles of Continuous Delivery and the release management lifecycle.
- **2. Manage Environments Effectively —** define and manage Dev, Test, UAT, and Production environments strategically.
- **3. Apply Deployment Strategies —** implement Blue-Green, Canary, Rolling, and Recreate deployment strategies.
- **4. Plan for Rollbacks and Recovery —** design rollback plans and apply disaster recovery best practices.
- **5. Implement Release Governance —** apply change management, approvals, and governance for safe releases.
- **6. Use Release Checklists and Quality Gates —** create and follow deployment checklists and quality gates to ensure readiness.
- **7. Automate and Optimize Releases —** use automation to streamline deployments and improve reliability.
- **8. Adopt Enterprise Best Practices —** apply proven practices to ensure safe, predictable, continuous value delivery.

KEY TAKEAWAY Goal: confidently deliver value to customers through automated, reliable, and governed releases. Deliver Value Faster. Reduce Risk. Ensure Quality. Build Trust.

WHAT IS CONTINUOUS DELIVERY?

Continuous Delivery (CD) is a DevOps practice where software changes are automatically built, tested, and prepared for release to production.

CD vs CONTINUOUS DEPLOYMENT

Continuous Delivery	Continuous Deployment
Automated up to production, but a human approves the final deploy.	Every change that passes all tests deploys to production automatically.

KEY BENEFITS

- **Faster Time to Market —** deliver features and fixes to users quickly.
- **Higher Quality —** automated testing and validation reduce defects.
- **Lower Risk —** small, frequent releases are easier to manage and roll back.
- **Better Collaboration —** improves transparency and alignment across teams.

The goal: keep software in a release-ready state, safe to release to production at any time.

THE CONTINUOUS DELIVERY PIPELINE

Commit -> automated build -> automated tests -> package & version -> auto-deploy to staging -> manual sign-off -> released to production.

KEY TAKEAWAY In short: Continuous Delivery ensures your software is always ready to release, providing speed, reliability, and confidence in every delivery -- with monitoring and feedback closing the loop.

What is Continuous Delivery?

Continuous Delivery (CD) is a software engineering practice where code changes are automatically **built, tested**, and **prepared for release** to production at any time.

It ensures that software is always in a deployable state.

THE CONTINUOUS DELIVERY PIPELINE

KEY BENEFITS

Faster Time to Market
Deliver features and fixes quickly and consistently.

Higher Quality
Automated testing and quality gates reduce defects.

Reliability & Consistency
Repeatable process ensures dependable releases.

Visibility & Confidence
Full visibility across the pipeline builds confidence to release.

Always Deployable
Code is always in a releasable state, reducing last-minute risks.

CONTINUOUS DELIVERY vs CONTINUOUS DEPLOYMENT

CONTINUOUS DELIVERY

- Code is always ready for release
- Deployment to production is manual (decision by a person)
- Adds a checkpoint before production

CONTINUOUS DEPLOYMENT

- Every change that passes tests is automatically released to production
- No manual approval needed
- Requires high confidence and automation

PREREQUISITES

- Automated Builds
- Automated Testing
- Version Control
- Infrastructure as Code
- Monitoring & Feedback

TRY IT YOURSELF — EXERCISE 1: CD vs CONTINUOUS DEPLOYMENT

Reinforces: the Continuous Delivery vs Continuous Deployment distinction you just saw, applied to Northstar CRM's own release path.

STEPS (notes/lab44-cd-vs-cdeploy.md)

Step	What To Do
1 — Definitions	Write two sentences: continuous delivery (always releasable) vs continuous deployment (auto-prod).
2 — Check the Reference	Confirm this cohort emphasizes delivery with gates and approvals -- not blind auto-prod.
3 — CRM Example	Describe promoting crm-api's digest that passed staging smoke for CUS-1001.
4 — Quiz Yourself	Answer: if staging said GO on digest X, what must production receive?

Reference: CD vocabulary

```
Continuous delivery      -> Main stays releasable; promote with gates.
Continuous deployment    -> Every green build may auto-deploy to prod.
Immutable identity       -> Identified by digest/checksum, never :latest.
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-cd-vs-cdeploy.md -> notes/lab44-cd-vs-cdeploy.md (workspace: examples/module-44-exercises).

CONTINUOUS DELIVERY LIFECYCLE

A structured, repeatable process that automates the path from code commit to production release -- delivering value quickly, safely, and reliably.

THE 8-STEP CD LIFECYCLE

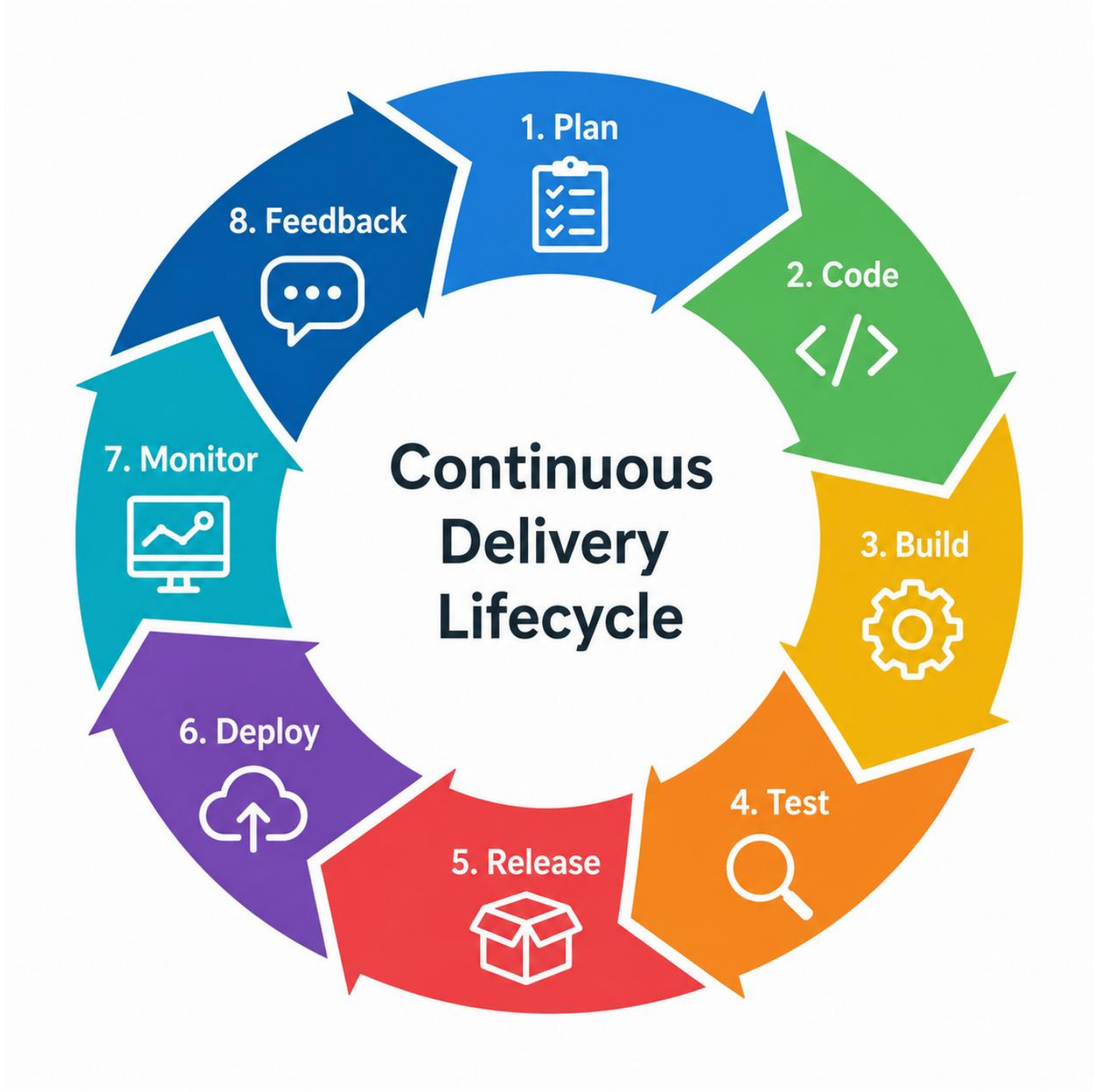
KEY PRINCIPLES

- **Automate Everything —** automate build, test, and deploy end-to-end.
- **Quality Built-In —** validate quality at every stage, not just at the end.
- **Collaborate —** work together across teams and tools, not in silos.
- **Continuously Improve —** measure, learn, and optimize constantly.
- **Release Safely —** use proven deployment strategies, checks, and rollbacks.

OUTCOME

Faster Delivery **Higher Quality** **Lower Risk** **Happier Customers**

KEY TAKEAWAY Deliver the right software to the right environment at the right time with confidence -- faster delivery, higher quality, lower risk, and happier customers.



RELEASE MANAGEMENT OVERVIEW

Release Management is the process of planning, scheduling, building, testing, deploying, and monitoring software releases in a controlled, repeatable manner.

OBJECTIVE

Deliver the right software, to the right environment, at the right time -- safely, reliably, and efficiently.

KEY STAKEHOLDERS

- **Business —** defines goals, priorities, and acceptance.
- **Product Management —** plans releases and manages backlog and scope.
- **Development —** builds, codes, and integrates features and fixes.
- **Quality Assurance —** tests, validates, and ensures quality standards.
- **Operations / DevOps —** deploys, monitors, and ensures system stability.
- **End Users —** receive value, provide feedback, and drive adoption.

SCOPE OF RELEASE MANAGEMENT

Planning | **Scheduling** | **Build & Package** | **Testing**

Deployment | **Monitoring**

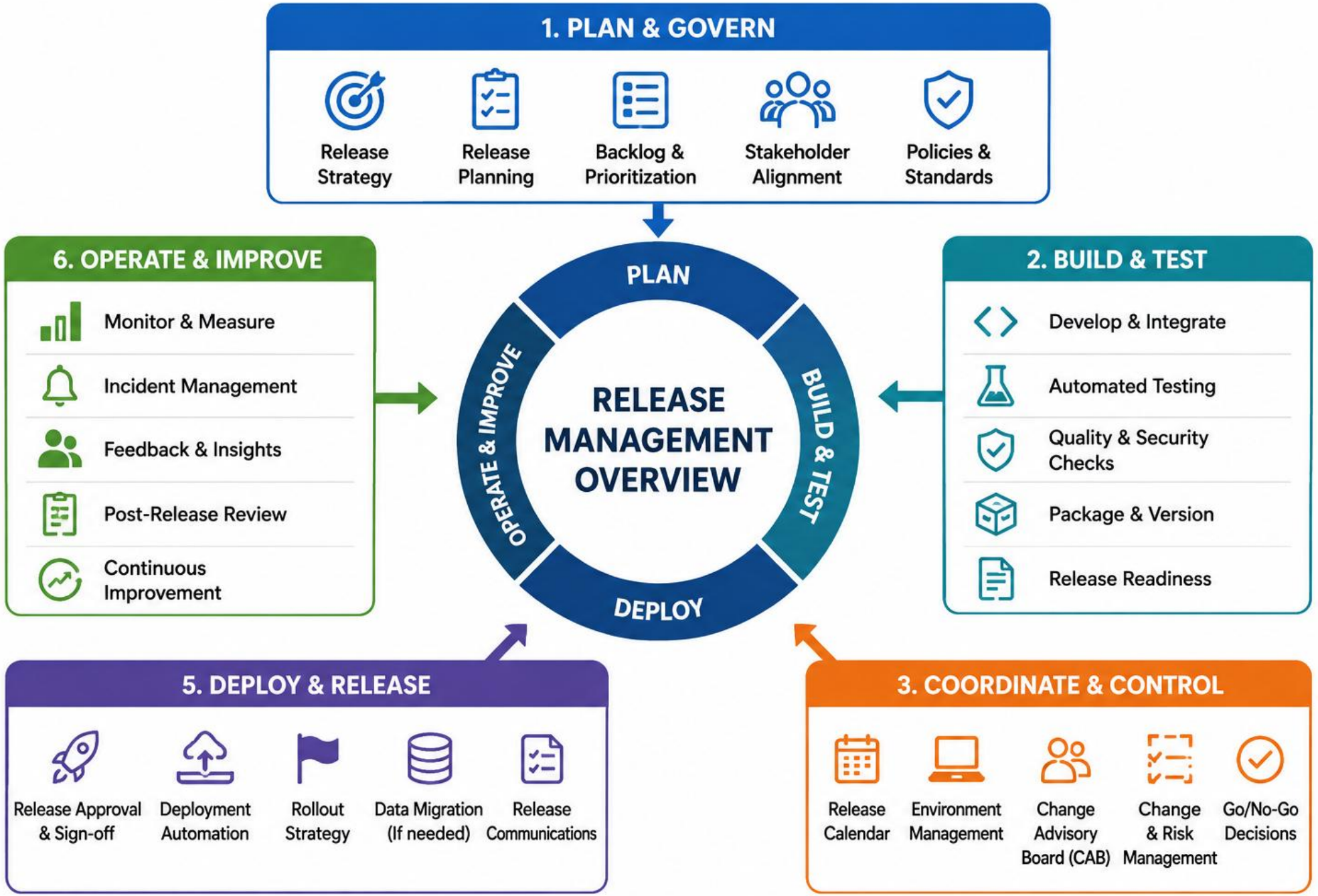
WHY IT MATTERS

- **Reduces Risk —** minimizes failures and service disruptions.
- **Faster Time to Value —** streamlined releases deliver features and fixes quickly.
- **Improves Predictability —** a standardized process ensures consistent outcomes.
- **Higher Quality —** built-in quality gates and testing improve reliability.
- **Enhances Collaboration —** aligns teams, tools, and processes across the org.
- **Customer Satisfaction —** stable, reliable releases improve trust and experience.

RELEASE MANAGEMENT IN THE DEVOPS LIFECYCLE

Plan -> Code -> Build -> Test (Development Pipeline) | **Release -> Deploy (Release Management)** | **Operate -> Monitor (Operations & Monitoring)**

KEY TAKEAWAY Bottom line: effective release management ensures every change is delivered as value -- with confidence, control, and continuous improvement. Plan well. Test thoroughly. Release safely.



TRY IT YOURSELF — EXERCISE 2: SKETCH ARTIFACT MANIFEST

Reinforces: the immutable-artifact identity theme from Release Management Overview, applied to Lab 44's own artifact-manifest.json.

STEPS (notes/lab44-manifest-fields.md)

Step	What To Do
1 — List Fields	List the fields for artifact-manifest.json: semver, git_sha, jar_sha256, image_digest, built_at, pipeline_run_url.
2 — Check the Reference	Confirm the prod candidate must match the staging digest exactly -- never rebuild per environment.
3 — Sample JSON	Write a JSON stub with placeholder digests and version 1.4.0-rc.1.
4 — Rollback Target	Add a known_good_previous example: version 1.3.2 plus a placeholder digest.

artifact-manifest.json stub

```
{
  "version": "1.4.0-rc.1", "gitSha": "<commit-sha>",
  "jarSha256": "<calculated-value>", "imageDigest": "sha256:<registry-digest>",
  "knownGoodPrevious": { "version": "1.3.2", "imageDigest": "sha256:<prior-digest>" }
}
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-manifest-fields.md -> notes/lab44-manifest-fields.md (workspace: examples/module-44-exercises).

DEVELOPMENT ENVIRONMENT

The Development Environment is where developers write, build, and integrate code -- optimized for speed, collaboration, and experimentation.

PURPOSE

Enable developers to build, unit test, and integrate code frequently in a safe, isolated space.

KEY CHARACTERISTICS

- **Rapid Iteration —** develop and test features quickly without impacting other environments.
- **Collaborative —** shared by the development team for coding, debugging, and peer review.
- **Isolated Data —** uses mock data or a local database that can be refreshed or reset at any time.
- **Flexible & Lightweight —** easy to set up and reconfigure; may not reflect the exact production setup.

BEST PRACTICES

- Keep it close to developers for fast feedback.
- Automate builds and unit tests.
- Use version control for all code changes.
- Refresh or reset data regularly.
- Promote only tested and stable code to lower environments.

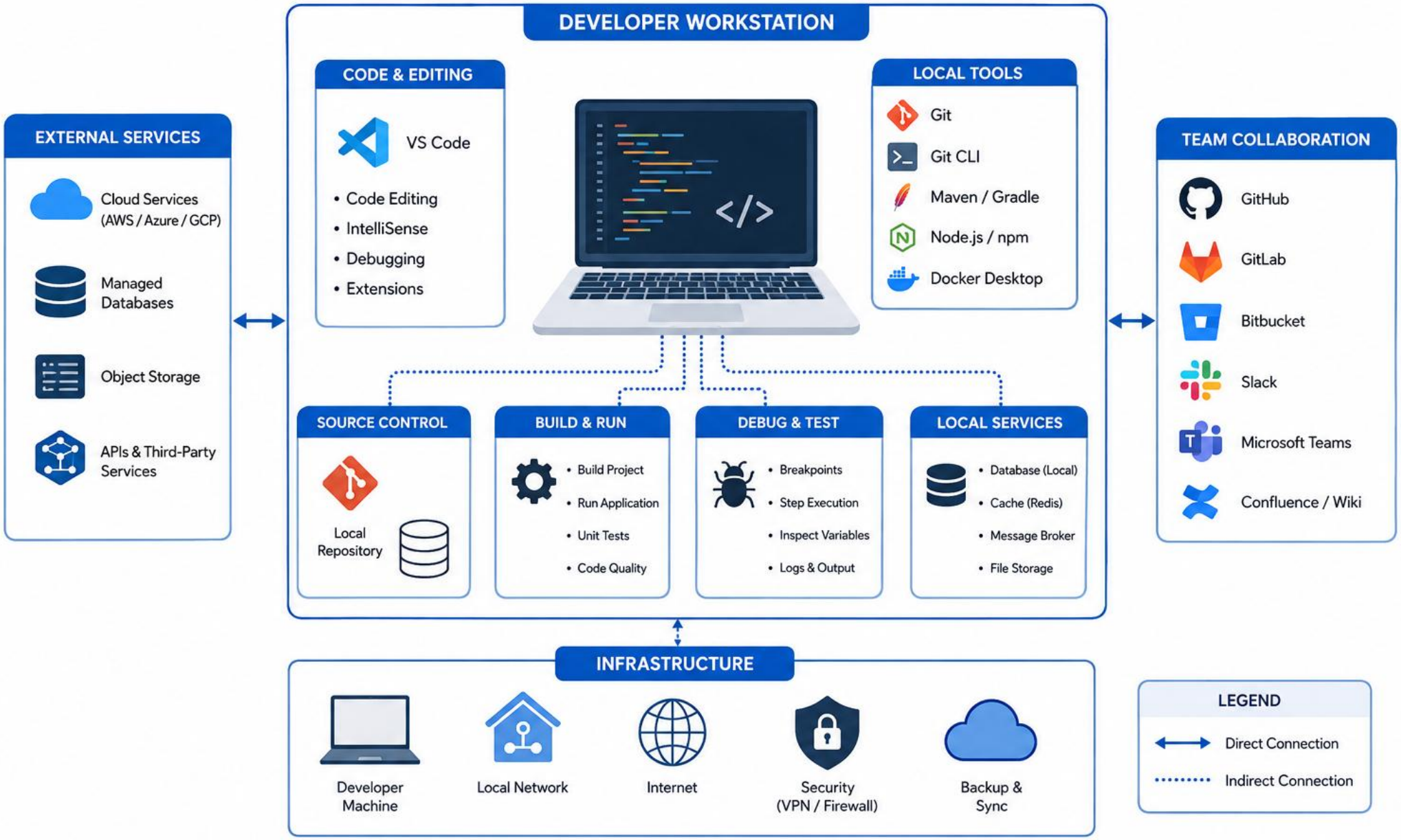
EXAMPLE TOOLING

IntelliJ / VS Code | **Docker Compose** | **Local PostgreSQL / H2**

Spring Boot dev profile

KEY TAKEAWAY The Development Environment is your sandbox for innovation -- build fast, test early, and integrate often.

DEVELOPMENT ENVIRONMENT



TESTING ENVIRONMENT

The Testing Environment is a dedicated, isolated environment used to validate quality, functionality, performance, and security before UAT or Production.

PURPOSE

Ensure the application works as expected, meets requirements, and is free of critical defects before release.

KEY CHARACTERISTICS

- **Test Coverage —** supports unit, integration, system, regression, performance, and security testing.
- **Stable & Repeatable —** environment configurations are consistent and can be recreated on demand.
- **Data Management —** uses masked or synthetic test data that represents real-world scenarios.
- **Isolated —** separate from Development and Production to avoid impact and data contamination.

BEST PRACTICES

- Mirror production as closely as possible.
- Automate test execution in the pipeline.
- Maintain test environment as code (IaC).
- Regularly refresh and clean test data.
- Track defects and ensure timely resolution.

TYPES OF TESTING SUPPORTED

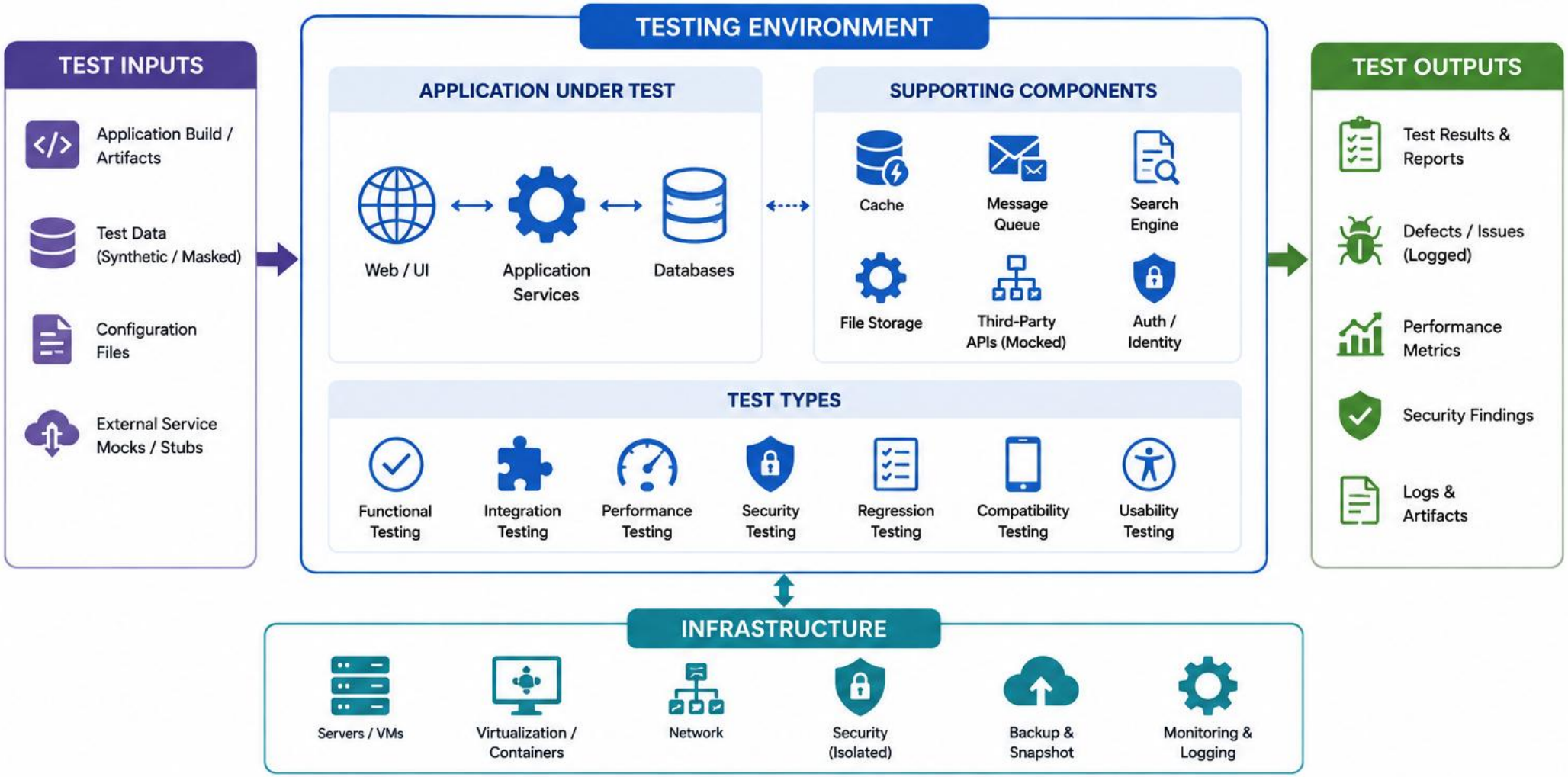
Unit **Integration** **System / E2E** **Regression**

Performance **Security**

KEY TAKEAWAY The Testing Environment gives confidence that the application is quality-checked, reliable, and ready for real users.

TESTING ENVIRONMENT

Isolated environment for functional, integration, performance and security testing



UAT ENVIRONMENT

The User Acceptance Testing (UAT) Environment mirrors production closely and is used by end users and business stakeholders to validate business requirements.

PURPOSE

Validate the application from an end-user and business perspective to confirm it meets requirements and is ready for production use.

KEY CHARACTERISTICS

- **Business-Focused Validation —** end users validate real business scenarios and workflows.
- **Production-Like —** configuration, data, integrations, and security mirror production as closely as possible.
- **Stable & Controlled —** the environment is stable, secure, and refreshed periodically with realistic data.
- **Defect Management —** issues identified are logged, tracked, and resolved before production release.

BEST PRACTICES

- Mirror production configurations and integrations.
- Use realistic, masked business data.
- Ensure environment stability and restricted changes.
- Provide clear UAT test cases and acceptance criteria.
- Obtain formal sign-off before production release.

COMMON ACTIVITIES

End-to-end validation

UI / usability checks

Role-based access checks

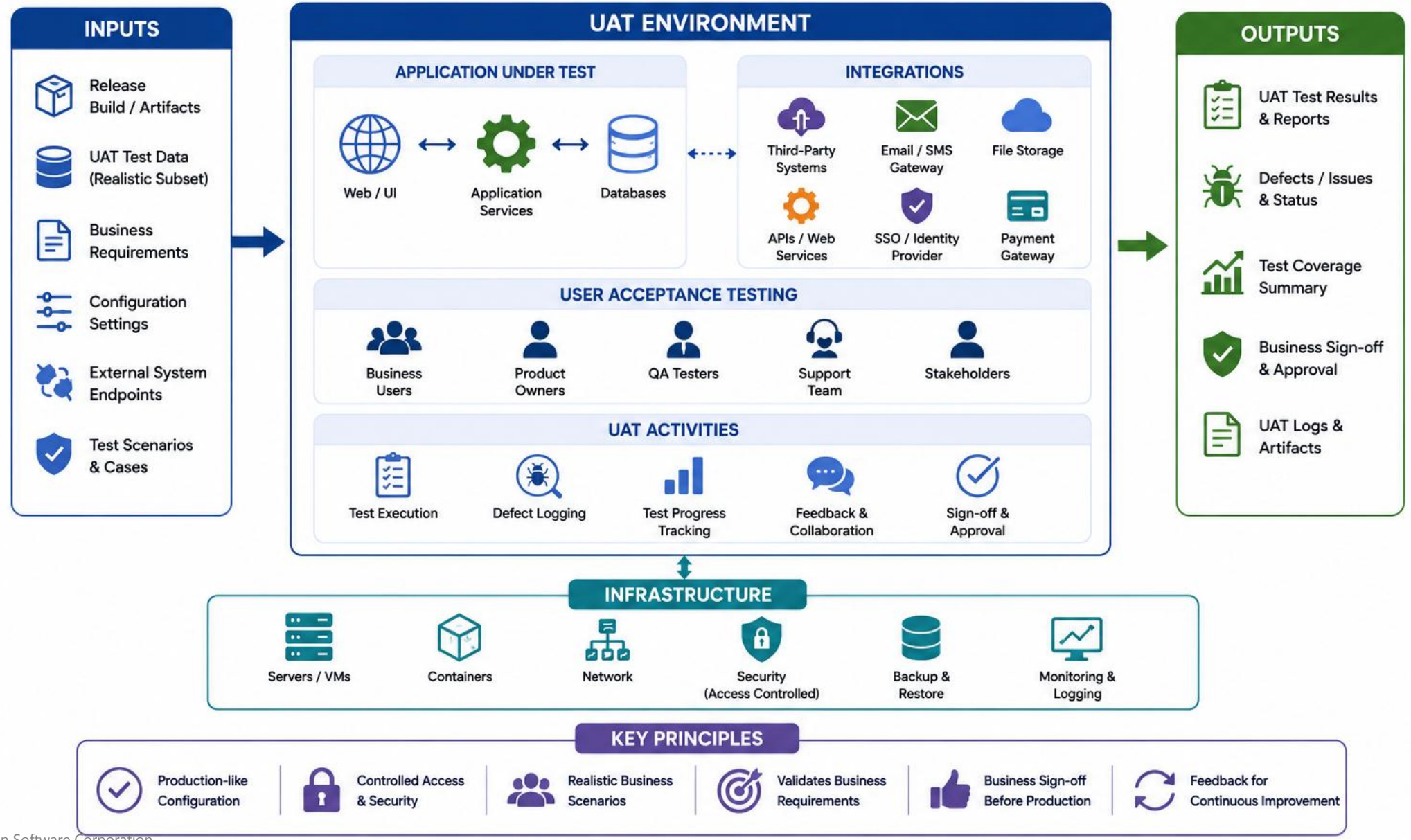
Data integrity checks

Final business sign-off

KEY TAKEAWAY The UAT Environment is the last checkpoint where business users validate the solution in a production-like setting before go-live.

UAT ENVIRONMENT

User Acceptance Testing in a production-like environment



PRODUCTION ENVIRONMENT

The Production Environment is the live environment where the application is available to end users -- highly available, secure, scalable, and monitored 24/7.

PURPOSE

Deliver the application to real users securely, reliably, and with optimal performance to meet business SLAs.

KEY CHARACTERISTICS

- **High Availability —** designed for uptime with redundancy, failover, and disaster recovery.
- **Scalability —** handles real user load and scales horizontally or vertically as needed.
- **Security —** hardened infrastructure, encryption, access control, and compliance.
- **Monitoring & Alerting —** continuously monitored for health, performance, and availability.

BEST PRACTICES

- Implement redundancy and failover.
- Automate deployments and infrastructure.
- Use feature flags for safe releases.
- Continuously monitor and alert.
- Perform regular backups and disaster recovery tests.

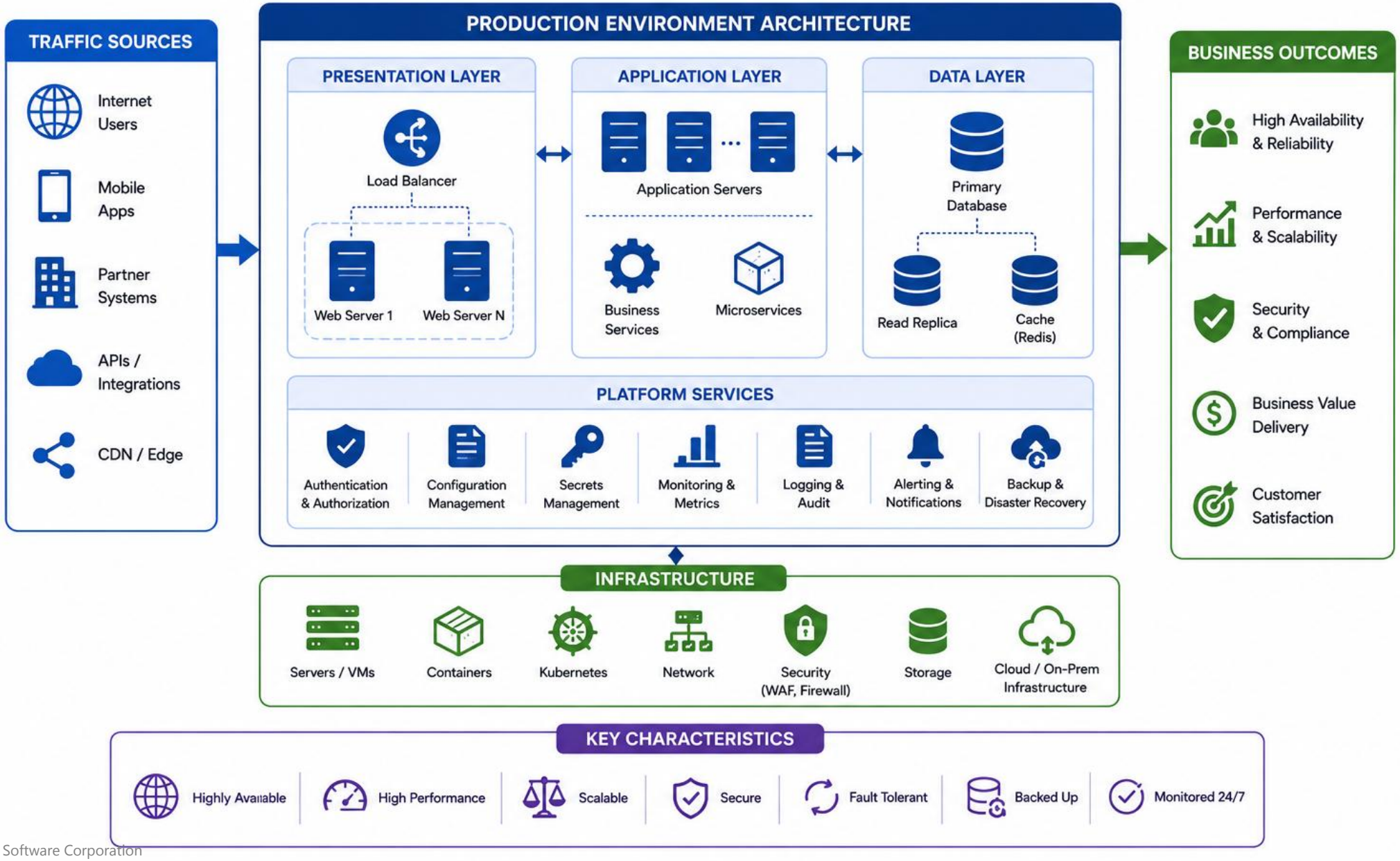
KEY FOCUS AREAS

User Experience | **Reliability & Uptime** | **Security & Compliance** | **Cost Efficiency**

KEY TAKEAWAY The Production Environment is the heart of your application delivery -- focus on reliability, security, scalability, and observability to deliver continuous value.

PRODUCTION ENVIRONMENT

Live environment delivering business value to end users



BLUE-GREEN DEPLOYMENT

Blue-Green Deployment maintains two identical environments -- Blue (Live) and Green (Staging) -- and switches traffic only after the new version is validated.

PURPOSE

Deliver new versions to production safely, quickly, and with low risk by keeping two identical environments.

KEY BENEFITS

- Minimal Downtime — traffic switch is instant.
- Low Risk — easy rollback, switch traffic back.
- Faster Releases — frequent, reliable deploys.
- Better User Experience — no partial updates.

BEST PRACTICES

- Keep Blue and Green environments identical.
- Automate deployment and validation.
- Run comprehensive tests before the switch.
- Use health checks and monitoring.
- Have a quick rollback plan; decommission the old environment to save costs.

WHEN TO USE

Web apps & APIs | **User-facing apps**

Frequent releases | **Quick rollback critical**

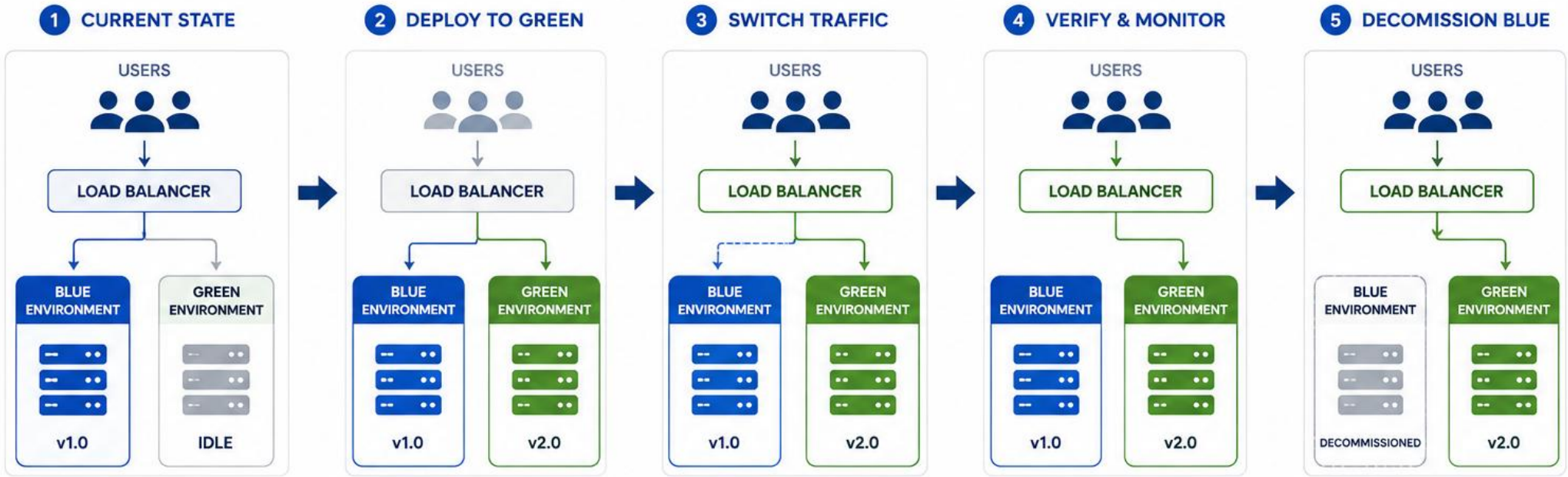
DEPLOYMENT FLOW

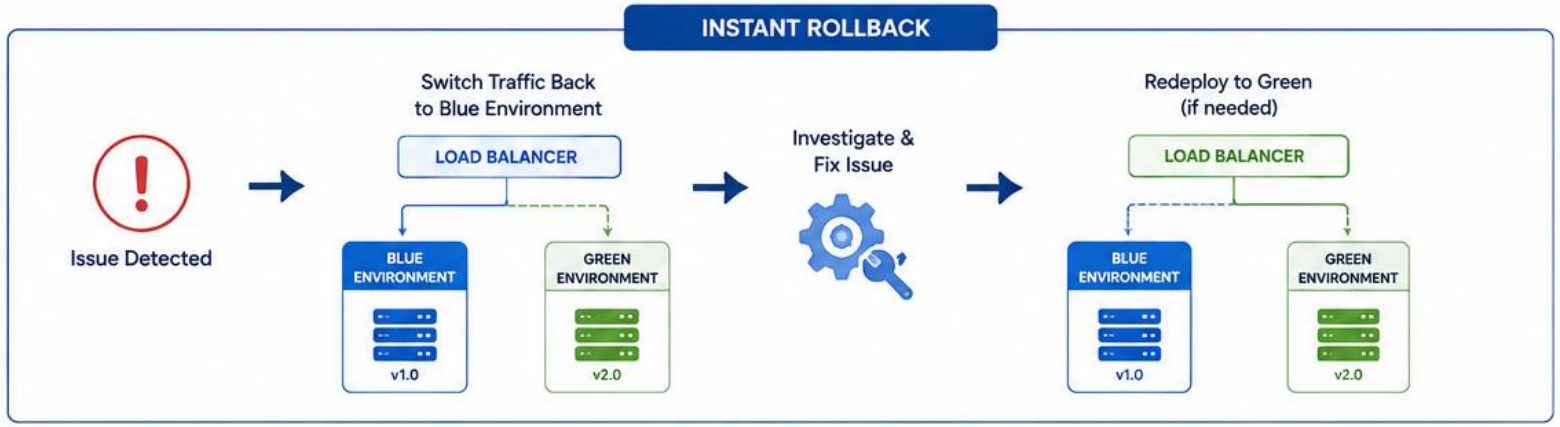
1. **Current State** — Blue is live.
2. **Deploy to Green** — New version deployed.
3. **Validate** — Automated & manual tests.
4. **Switch Traffic** — Route Blue -> Green.
5. **Retire Blue** — Old env decommissioned.

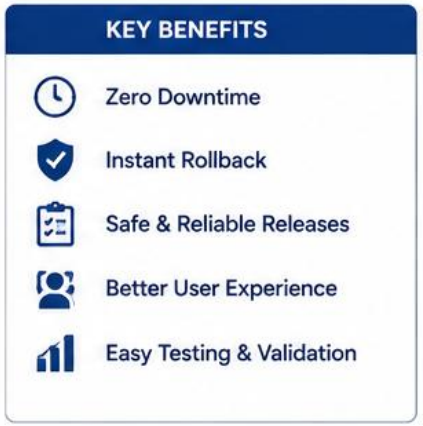
KEY TAKEAWAY Blue-Green Deployment reduces risk and downtime by keeping two identical environments and switching traffic only after the new version is validated.

BLUE-GREEN DEPLOYMENT

Zero-downtime deployment with instant rollback







CANARY DEPLOYMENT

Canary Deployment rolls out a new version to a small subset of users first; if it performs well, traffic increases gradually until it reaches 100%.

PURPOSE

Validate a new version in production with real user traffic before exposing it to all users, minimizing risk.

KEY BENEFITS

- Reduced Risk — only a small user group affected.
- Early Detection — monitoring catches issues early.
- Fast & Safe — ship updates without instability.
- Data-Driven — metrics decide rollout or rollback.

BEST PRACTICES

- Start with a very small traffic percentage (1-5%).
- Use automated monitoring and alerting.
- Define clear success metrics and thresholds.
- Increase traffic gradually and in stages.
- Have an automated rollback strategy; test in non-production first.

WHEN TO USE

Web & mobile apps | **Microservices**

APIs & back-end services

Frequent releases

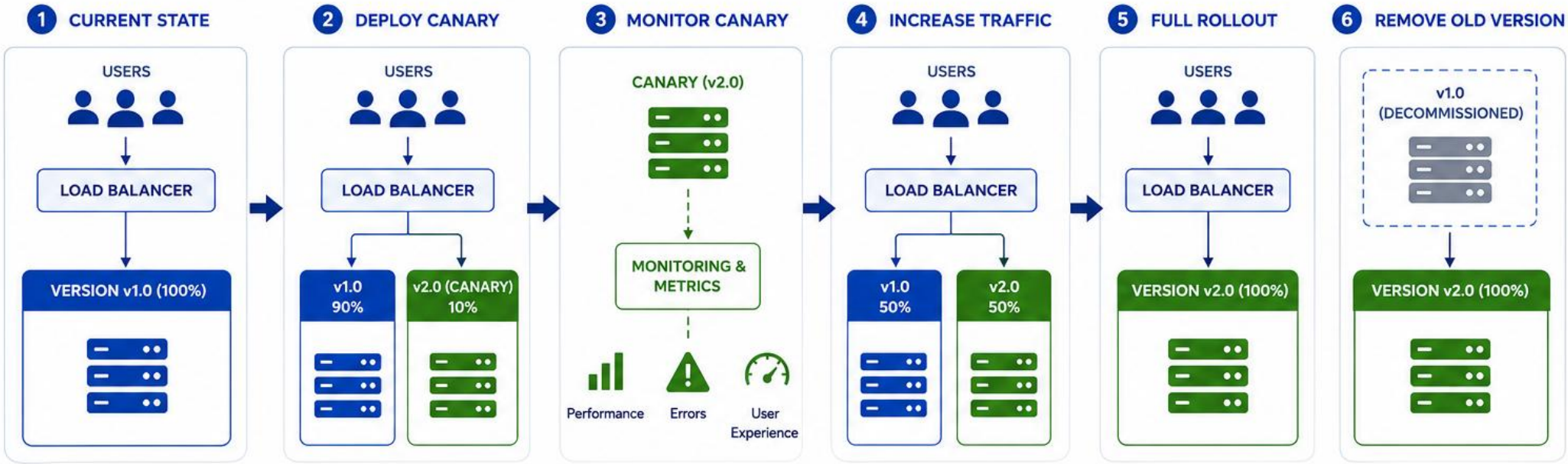
DEPLOYMENT FLOW

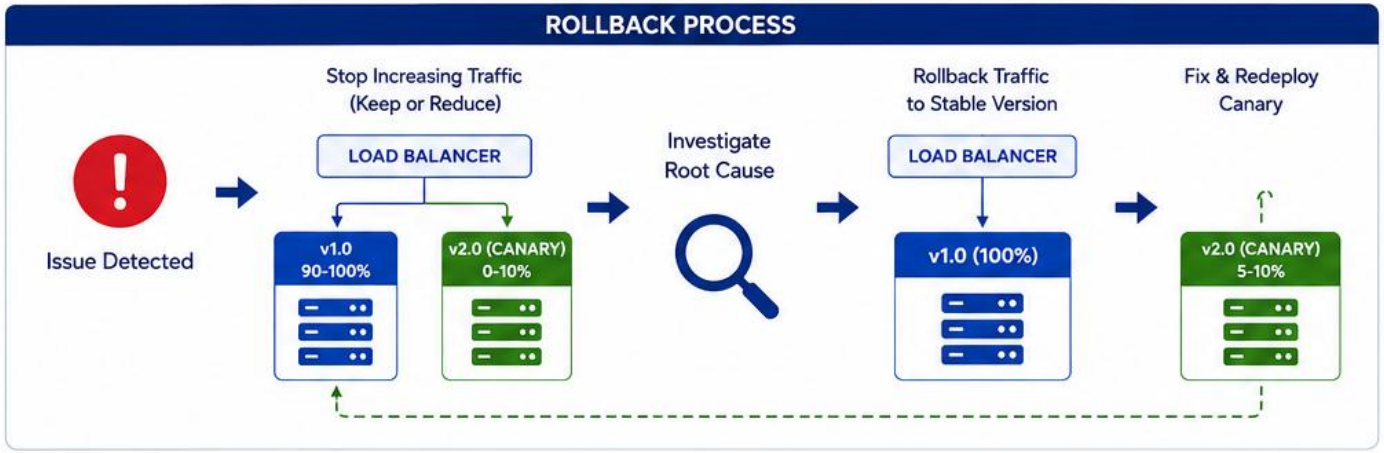
1. **Deploy Canary**
   Small subset, e.g. 5%.
2. **Monitor & Validate**
   Logs, metrics, alerts.
3. **Increase Traffic**
   10% -> 25% -> 50% -> 100%.
4. **Full Rollout**
   New version becomes stable.

KEY TAKEAWAY Canary Deployment lets you release with confidence by validating in production with a small user group first, minimizing risk and improving stability.

CANARY DEPLOYMENT

Gradual rollout to a subset of users to reduce risk





ROLLING DEPLOYMENT

Rolling Deployment gradually replaces the old version of an application with the new version, one instance or server at a time, with zero downtime.

PURPOSE

Deploy new versions with zero downtime by updating instances in a rolling manner while maintaining desired capacity.

KEY BENEFITS

- Zero Downtime — healthy instances serve users.
- Gradual Risk — issues detected early.
- Auto Recovery — unhealthy instances roll back.
- Resource Efficient — no duplicate environments.

BEST PRACTICES

- Use health checks to validate instances.
- Deploy during low-traffic periods when possible.
- Monitor metrics (CPU, memory, latency, errors).
- Use small batch sizes for high-risk releases.
- Automate rollback on failure; test in staging before production.

WHEN TO USE

Stateless apps & microservices

Web apps & APIs **Zero downtime required**

Kubernetes / Docker Swarm

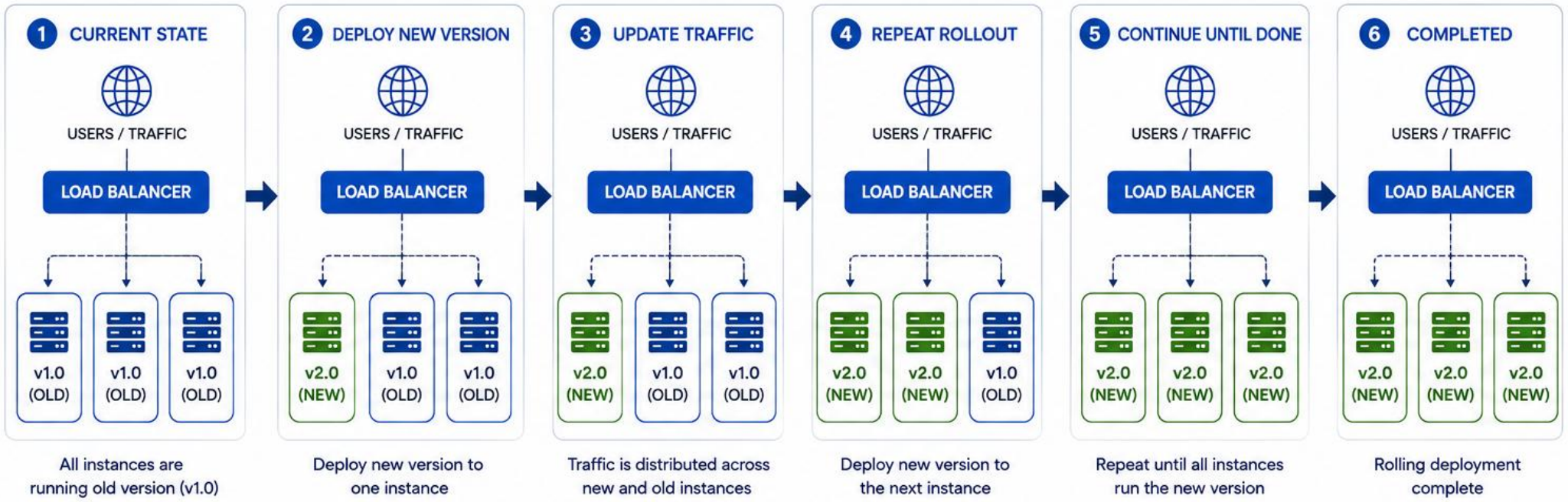
DEPLOYMENT FLOW

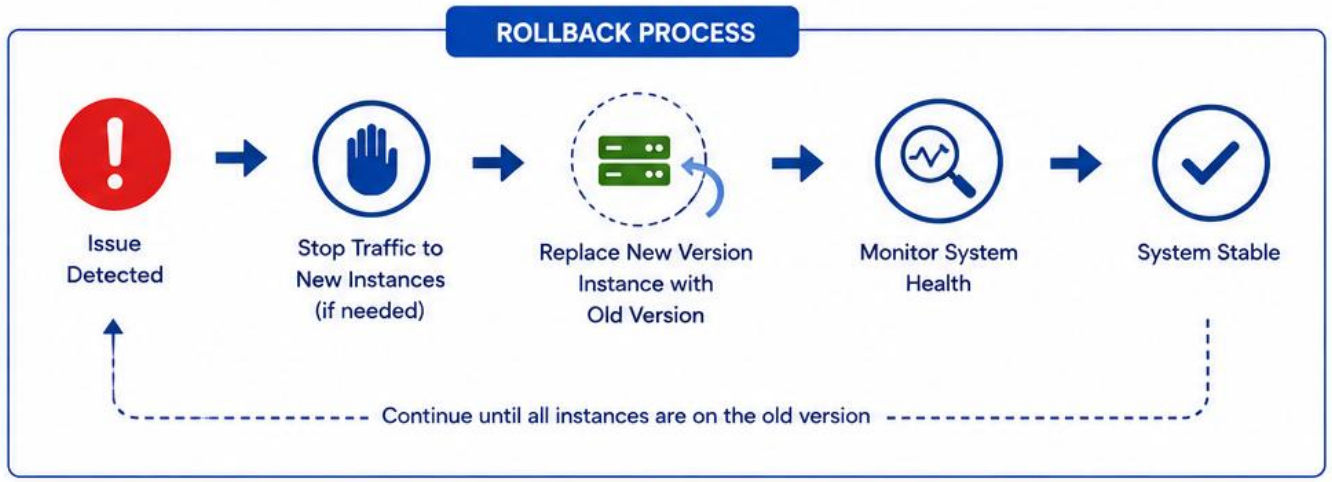
1. **Current State** — 100% old version (v1).
2. **Update Instance 1** — 25% updated.
3. **Update Instance 2** — 50% updated.
4. **Update Instance 3** — 75% updated.
5. **All Updated** — 100% new version (v2) live.

KEY TAKEAWAY Rolling Deployment updates your application gradually, one instance at a time, ensuring continuous availability and fast recovery if something goes wrong.

ROLLING DEPLOYMENT

Gradually replace old version with new version, one instance at a time





RECREATE DEPLOYMENT

Recreate Deployment completely shuts down existing application instances before starting the new version -- a clean replacement that accepts downtime.

PURPOSE

Replace all application instances with the new version by terminating existing instances first, ensuring a clean, consistent deployment.

KEY BENEFITS

- Full Replacement — old instances stop first.
- Downtime Possible — app unavailable during update.
- No Version Overlap — old/new never run together.
- Consistent State — all instances match exactly.

BEST PRACTICES

- Use during low-traffic periods and notify users of expected downtime.
- Automate the deployment and health checks.
- Keep the previous version available for a quick rollback.
- Validate the new version thoroughly before go-live.
- Monitor application health closely after deployment.

WHEN TO USE

- **App state can't be maintained**
- **DB schema needs full restart**
- **Incompatible config changes**
- **Major upgrades / legacy systems**

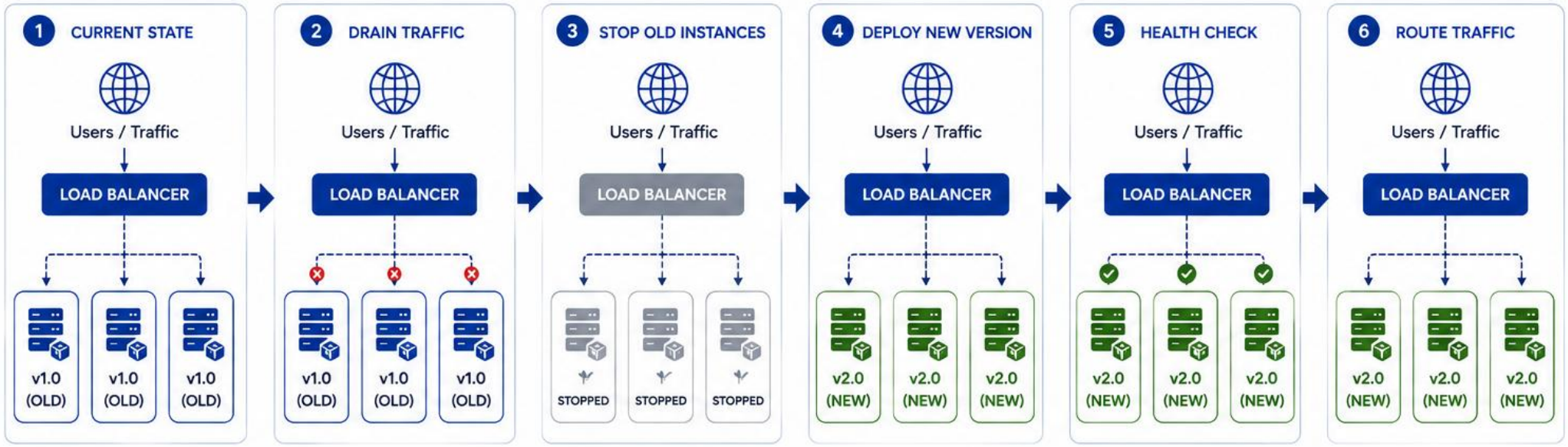
DEPLOYMENT FLOW

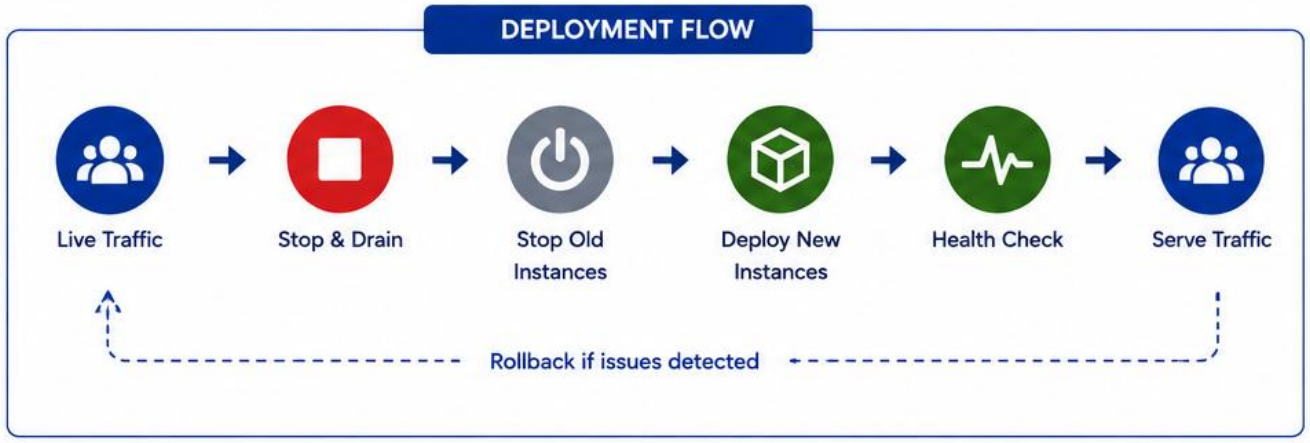
1. **Stop All Instances** — Terminate old version (v1).
2. **Deploy New Version** — Start instances with v2.
3. **Health Check** — Verify app is running.
4. **Validate** — Smoke tests, monitor.
5. **Go Live** — New version serving traffic.

KEY TAKEAWAY Recreate Deployment ensures a simple, consistent update by replacing all instances at once -- simplicity and predictability in exchange for accepted downtime.

RECREATE DEPLOYMENT

Stop the old version and start new version (full replacement)





COMPARING DEPLOYMENT STRATEGIES

Choosing the right deployment strategy depends on downtime tolerance, rollback speed, infrastructure cost, and how much risk you can accept per release.

Strategy	Downtime	Rollback Speed	Infra Cost	Best For
Blue-Green	None (traffic switch)	Instant	Higher (2 environments)	High-availability apps
Canary	None (gradual rollout)	Fast (route to 0%)	Moderate (extra capacity)	Microservices & APIs
Rolling	None (zero downtime)	Automatic, gradual	Low (no duplicate env)	Stateless apps, K8s
Recreate	Yes (full outage)	Redeploy previous (slower)	Lowest (1 environment)	Major upgrades, schema changes

QUICK DECISION GUIDE

- **Need instant rollback and can afford two environments? —** use Blue-Green.
- **Want to validate with real traffic gradually? —** use Canary.
- **Need zero downtime without doubling infrastructure? —** use Rolling.
- **Can accept downtime for a clean, simple cutover? —** use Recreate.

KEY TAKEAWAY There is no single best deployment strategy -- match the strategy to your downtime tolerance, rollback needs, and infrastructure budget.

WHY ROLLBACKS MATTER

Even with the best testing and monitoring, issues can reach production. Rollbacks give us a safe, fast way to restore stability and protect users.

THE REALITY OF PRODUCTION

- **Bugs Happen —** not all issues can be found in testing; real user scenarios are unpredictable.
- **Unexpected Impact —** performance degradation or dependency failures can affect users.
- **User Experience at Risk —** slow, broken, or unavailable features frustrate users and lose trust.
- **Business Impact —** incidents can cause revenue loss, SLA breaches, and brand damage.

WITH ROLLBACK vs WITHOUT ROLLBACK

With Rollback	Without Rollback
Quick recovery to last stable version.	Longer downtime and investigation.
Minimal downtime and disruption.	Higher impact on users and revenue.
Maintains user trust.	Risk of SLA breaches.
Confidence to release more often.	Slower delivery velocity.

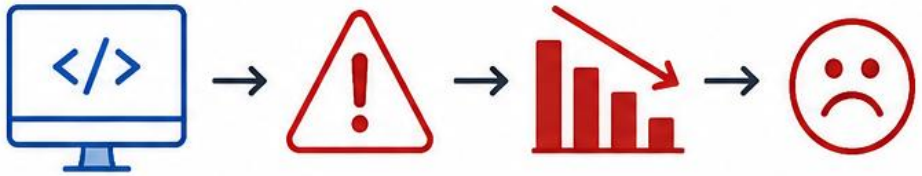
WHEN ROLLBACKS ARE ESSENTIAL

Critical Bugs | **Performance Issues** | **Integration Failures** | **Security Problems** | **Bad User Experience**

KEY TAKEAWAY Rollback is your safety net to act fast and reduce impact. Rollbacks are not a failure -- they are a sign of a resilient, well-prepared delivery process.

Why Rollbacks Matter

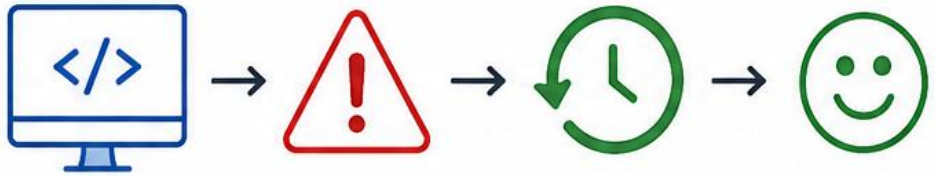
WITHOUT ROLLBACK



Deploy New Release → Issue or Bug Occurs → Downtime, Errors & Lost Users → Revenue Loss & Poor Experience

Without rollback, small issues become big problems.

WITH ROLLBACK



Deploy New Release → Issue or Bug Occurs → Rollback to Previous Version → System Restored, Users Happy

With rollback, you quickly recover and keep users and business safe.

RISKS WITHOUT ROLLBACK

- Longer downtime
- More errors and support tickets
- Loss of users and revenue
- Damage to reputation

BENEFITS WITH ROLLBACK

- Quick recovery from issues
- Minimal downtime
- Happy users
- Business continuity and trust

ROLLBACK STRATEGIES

A rollback strategy is your plan to quickly and safely return to a known good state when a deployment causes issues.

Strategy	Best For
Recreate / Redeploy Previous Version	Simple applications and stateless services.
Blue-Green Rollback	Applications with blue-green deployment.
Canary Rollback	Canary or iterative releases.
Rolling Update Rollback	Kubernetes, container, or orchestrated apps.
Database Rollback	When schema or data changes cause issues.

ROLLBACK PROCESS (HIGH LEVEL)

1. **Detect** — Monitor & alerts.
2. **Assess** — Validate & scope.
3. **Decide** — Choose strategy.
4. **Execute** — Auto or manual.
5. **Verify** — Confirm stability.
6. **Review** — Post-incident.

KEY TAKEAWAY A well-defined rollback strategy is as important as the deployment itself. Fast, tested rollbacks ensure you can recover quickly, minimize impact, and deliver with confidence.

Rollback Strategies

	1. Red/Blue (Blue-Green)	2. Canary Rollback	3. Rolling Rollback	4. Recreate Rollback	5. Database Rollback
					
How it Works	Run two identical environments. Switch traffic between versions.	Release to a small subset of users. Monitor, then decide to continue or rollback.	Gradually replace old instances with new ones.	Stop the new version and start the previous version.	Revert database changes to a previous state.
Rollback Process	Switch traffic back to the previous (blue) environment.	Stop the rollout and route all traffic back to the stable version.	Replace new instances with old instances one by one.	Terminate the new version and redeploy the old version.	Restore from backup or apply reverse migration.
Best Use Case	High-traffic applications needing instant rollback.	New features or versions with uncertain impact.	Large systems where zero downtime is required.	Small applications or when simplicity is preferred.	Schema changes, critical data modifications.
Pros	• Instant rollback • Zero downtime • Easy to implement	• Low risk • Real user feedback • Safe progression	• Zero downtime • Controlled rollout • Automatic rollback possible	• Simple and fast • Easy to understand • No extra infra needed	• Protects data integrity • Point-in-time recovery
Cons	• Double infrastructure • Higher cost	• Takes longer • Complex monitoring needed	• More complex • Rollback takes time • Requires automation	• Downtime during rollback • Not ideal for large apps	• Backup overhead • Data loss risk if backup is old

TRY IT YOURSELF — EXERCISE 5: OUTLINE ROLLBACK RUNBOOK

Reinforces: the Rollback Strategies you just saw, applied as a concrete runbook outline naming a known-good digest Y.

STEPS (notes/lab44-rollback-runbook.md)

Step	What To Do
1 — Order the Steps	Detect -> decide -> redeploy known-good digest -> verify readiness -> CRM smoke -> comms update.
2 — Check the Reference	Confirm the runbook names digest Y and a verification check -- never 'just redeploy latest'.
3 — Timebox It	Write a target recovery time placeholder (e.g. under N minutes) and who declares the SEV.
4 — Kafka Watch (Optional)	Add one line: watch consumer lag after rollback (full detail arrives in Lab 46).

Rollback runbook outline

```
# Rollback -- crm-api
Triggers: readiness failing > 3m, error rate above threshold
Authority: release commander decides; on-call executes
Steps: confirm digest != known-good -> promote knownGoodPrevious
       -> verify readiness + CUS-1001 / CUS-1002 smoke
Limits: non-backward-compatible migration -> stop, escalate
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-rollback-runbook.md -> notes/lab44-rollback-runbook.md (workspace: examples/module-44-exercises).

DISASTER RECOVERY BASICS

Disaster Recovery (DR) is the capability to quickly restore critical IT systems and data after a disruptive event, minimizing downtime and data loss.

KEY OBJECTIVES

- Minimize downtime and service disruption.
- Protect critical data and ensure integrity.
- Restore business operations quickly.
- Meet regulatory and compliance requirements.
- Maintain customer trust and business continuity.

DR STRATEGIES

Strategy	Approach
Backup & Restore	Restore from backups. Low cost, higher downtime.
Pilot Light	Minimal DR resources; scale up on disaster.
Warm Standby	Scaled-down prod copy, ready to take over.
Active-Active	Both sites live; traffic split continuously.

RECOVERY OBJECTIVES

RTO (Time)	RPO (Data Loss)
Max acceptable time to restore systems. Example: RTO = 4 hours.	Max acceptable data loss, measured in time. Example: RPO = 15 minutes.

BEST PRACTICES

- Keep the DR plan simple, documented, and current.
- Test the DR plan regularly (at least quarterly).
- Automate backups and failover where possible.
- Monitor systems 24/7 and continuously improve the strategy.

KEY TAKEAWAY Business continuity depends on recovery. Plan ahead, protect data, meet RTO/RPO, and test regularly -- disasters are unpredictable, but your recovery doesn't have to be.

Disaster Recovery Basics

1. WHAT IS DISASTER RECOVERY?

Disaster Recovery (DR) is a set of policies, tools, and procedures that restore IT systems and data after a disruptive event.

2. WHY IT MATTERS

- Minimize downtime
- Reduce financial impact
- Protect customers and reputation
- Ensure business continuity and compliance

3. KEY DR METRICS

RTO (Recovery Time Objective)
Maximum acceptable time to restore systems.

RPO (Recovery Point Objective)
Maximum acceptable data loss (measured in time).

4. DISASTER RECOVERY PROCESS

1. **Plan** — Identify critical systems, risks, and recovery goals.
2. **Protect** — Back up data and replicate systems as needed.
3. **Detect** — Detect an incident or disaster event.
4. **Recover** — Failover to recovery site and restore systems.
5. **Validate** — Verify systems and data are fully functional.
6. **Resume** — Return to normal operations and monitor performance.

5. DISASTER RECOVERY STRATEGIES

Backup & Restore
Restore systems and data from backups.
Lowest cost, higher RTO/RPO

Pilot Light
Keep core systems running in DR environment.
Balanced cost and recovery time

Warm Standby
Keep scaled-down systems running in DR environment.
Faster recovery, higher cost

Active-Active
Run fully redundant systems in both locations.
Highest availability, highest cost

6. BEST PRACTICES

- Keep DR plan simple, documented, and up to date.
- Test DR plan regularly.
- Assign roles and responsibilities.
- Ensure backups are reliable and verified.
- Monitor, review, and improve continuously.

CHANGE MANAGEMENT

Change Management is the structured approach to controlling the lifecycle of changes to IT systems, ensuring they are beneficial, low-risk, and traceable.

CHANGE MANAGEMENT PROCESS

1. **Request** — Justify need.
2. **Assessment** — Assess risk.
3. **Approval** — CAB approves.
4. **Implementation** — Execute safely.
5. **Review** — Verify outcome.
6. **Closure** — Capture lessons.

TYPES OF CHANGES

Type	Example
Standard	Pre-approved, low-risk (e.g., OS patch).
Normal	Needs assessment & approval (e.g., app upgrade).
Emergency	Urgent, immediate approval (e.g., security patch).

WHO'S INVOLVED

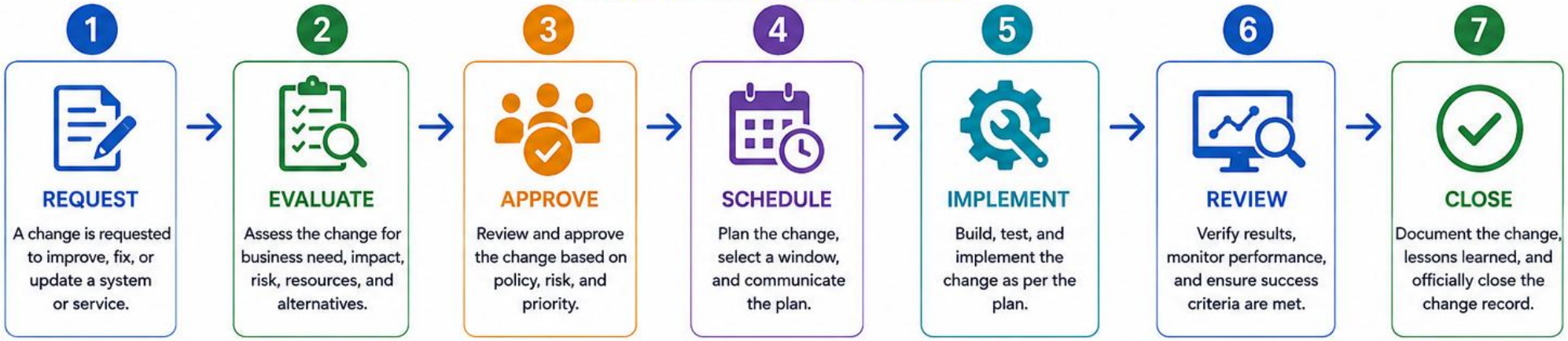
- **Change Requester —** raises the request with details.
- **Change Manager —** coordinates the process.
- **Change Advisory Board (CAB) —** reviews and approves changes.
- **Implementers & Reviewers —** execute and validate the change.

KEY TAKEAWAY Every change is a potential risk. Good change management turns risk into a controlled opportunity -- minimize risk, ensure quality, maintain transparency, and drive continuous improvement.

Change Management

Change Management is the process of controlling changes to IT systems and services in a way that minimizes risk and supports business goals.

CHANGE MANAGEMENT PROCESS



CHANGE TYPES

STANDARD
Pre-approved, low-risk changes with a defined process.

NORMAL
Changes that require review and approval.

EMERGENCY
Urgent changes to restore service or fix critical issues.

MAJOR
Large-scale changes with significant impact or risk.

KEY BENEFITS

- Reduces risk of outages and failures
- Improves service quality and reliability
- Ensures accountability and transparency
- Reduces cost of failed changes
- Supports compliance and audit readiness
- Aligns IT changes with business goals

BEST PRACTICES

- Have clear policies and procedures
- Communicate clearly and early
- Assess risk and impact carefully
- Schedule and plan changes properly
- Monitor and validate outcomes
- Continuously review and improve

Goal: Implement the right changes, the right way, at the right time.

Result: Stable systems, satisfied users, and successful outcomes.

RELEASE APPROVAL PROCESS

The Release Approval Process ensures every release is reviewed, validated, and authorized by the right stakeholders before deployment to production.

RELEASE APPROVAL WORKFLOW

1. **Ready** — Gates completed.
2. **Submit** — Artifacts sent.
3. **Review** — Stakeholders review.
4. **Decision** — Approve or reject.
5. **Notify** — Decision recorded.
6. **Deploy** — To pipeline.

APPROVAL LEVELS (EXAMPLE)

Environment	Approvers	Purpose
Dev / QA	QA Lead, Dev Lead	Validate quality & readiness.
Staging	QA, Security, Ops Leads	Validate production readiness.
Production	CAB, Business Owner	Final go/no-go for production.

KEY STAKEHOLDERS

- **Release Manager —** owns and coordinates the approval process.
- **QA Lead / Security Officer —** validate quality, risk, and compliance.
- **Business Owner —** confirms business value and user impact.
- **Change Advisory Board (CAB) —** provides final approval for production.

KEY TAKEAWAY A strong approval process is your last line of defense before production. Review thoroughly. Approve wisely. Deliver with confidence.

Release Approval Process

1 — Release Ready

Development and testing complete. Release is prepared with all artifacts and documentation.

2 — Submit for Approval

Release owner submits the release package and release notes for review.

3 — Review & Assess

Approvers review the release for quality, risk, and compliance.

4 — Impact & Risk Evaluation

Assess business impact, technical risk, dependencies, and rollback plan.

5 — Approval Decision

Approvers record their decision: Approve, Reject, or Request Changes.

6 — Resolve & Re-Submit

Address feedback or issues and re-submit for approval.

7 — Final Approval

All required approvals are obtained. Release is authorided.

8 — Proceed to Release

Release moves to deployment as per the approved plan and schedule.

APPROVAL ROLES

Release Manager
Ensures the release is ready and complete.

Quality Assurance
Verifies test results and quality standards.

Security
Reviews security requirements and compliance.

Operations
Validates operational readiness and support.

Business Owner
Confirms business value and accepts the release.

APPROVAL DECISIONS

Decision	Description	Action
Approve	Release meets all criteria and is approved to proceed.	Move to final approval and release deployment.
Reject	Release does not meet criteria or has high risk.	Release is stopped. Issues must be resolved.
Request Changes	Changes or additional information are required.	Address feedback and re-submit for approval.

Approval Criteria

- Functionality and quality requirements met
- Security and compliance standards met
- Operational readiness and support in place
- Rollback plan and communication plan ready
- Business impact and go/no-go assessed

KEY BENEFITS

Ensures quality and compliance

Reduces risk of failed releases

Improves collaboration and accountability

Aligns releases with business goals

Increases release success and stability

TRY IT YOURSELF — EXERCISE 3: DEFINE PROMOTION GATES

Reinforces: the Release Approval Process you just saw, applied as measurable, owned promotion gates for Northstar CRM.

STEPS (notes/lab44-promotion-gates.md)

Step	What To Do
1 — Gate List	List gates: verify green, SAST scan, staging smoke, change approval, residual risk owned.
2 — Check the Reference	Confirm gates need evidence links, not vibes -- 'QA feels good' is not a gate.
3 — Owner Column	Assign a role owner to each gate: QA lead, dev lead, security, or ops.
4 — No-Go Examples	List three automatic no-go conditions: secret leak, digest mismatch, failed readiness.

Gate table sketch

```
Gate                          Owner       Evidence
CI verify green               Dev Lead    pipeline run URL
Security scan clean           Security    scan report link
Staging smoke (CUS-1001/1002) QA          screenshot / log
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-promotion-gates.md -> notes/lab44-promotion-gates.md (workspace: examples/module-44-exercises).

DEPLOYMENT CHECKLISTS

Deployment checklists help teams follow best practices, reduce errors, and ensure consistent, repeatable, successful releases across every environment.

1. Pre-Deployment

Validate readiness: tests passed, build verified, rollback plan documented, stakeholders notified.

2. Deployment Execution

Follow deployment steps closely: environment verified, backup taken, health checks passed, no errors.

3. Post-Deployment

Validate deployment: smoke tests passed, monitoring reviewed, error rates checked, docs updated.

4. Rollback Readiness

Ensure rollback plan is ready: prior version available, backups verified, team on standby.

5. Post-Deployment Review

Review and capture learnings: metrics reviewed, lessons learned, checklist updated for next time.

BEST PRACTICES

- **Keep checklists simple & role-based**
- **Automate as much as possible**
- **Review and update regularly**
- **Make it part of the workflow**

KEY TAKEAWAY A consistent checklist today prevents incidents tomorrow -- and builds a culture of quality and reliability. Reduce Risk. Save Time. Improve Quality. Build Confidence.

Deployment Checklists

1. PRE-DEPLOYMENT

- [x] Review requirements and scope
- [x] Verify change request approval
- [x] Confirm deployment window
- [x] Communicate plan to stakeholders
- [x] Backup current system/data
- [x] Verify access and permissions
- [x] Check dependencies and integrations
- [x] Ensure rollback plan is ready

2. BUILD & TEST

- [x] Code committed and reviewed
- [x] Build successful
- [x] Unit tests passed
- [x] Integration tests passed
- [x] Security scan completed
- [x] Performance tests passed
- [x] Artifacts/version validated
- [x] Test environment verification

3. PRE-DEPLOY CHECKS

- [x] Verify target environment readiness
- [x] Check system requirements
- [x] Verify configuration settings
- [x] Confirm database migrations (if any)
- [x] Validate feature flags and toggles
- [x] Check third-party services
- [x] Confirm monitoring and alerting
- [x] Final smoke test in staging

4. DEPLOYMENT

- [x] Deploy to target environment
- [x] Monitor deployment progress
- [x] Verify services started successfully
- [x] Run smoke tests
- [x] Validate critical functionality
- [x] Check logs for errors
- [x] Confirm data integrity
- [x] Update deployment status

5. POST-DEPLOYMENT

- [x] Monitor system health
- [x] Verify performance metrics
- [x] Check error rates and alerts
- [x] Validate integrations and dependencies
- [x] Confirm business processes
- [x] User acceptance verification
- [x] Update documentation
- [x] Communicate completion

6. ROLLBACK READINESS

- [x] Verify rollback plan is accessible
- [x] Validate backup availability
- [x] Test rollback procedure (if needed)
- [x] Confirm rollback criteria
- [x] Document known issues
- [x] Ensure team on-call readiness
- [x] Review and learn from deployment

BEST PRACTICES

Plan Ahead
Communicate Early

Automate What
You Can

Monitor
Continuously

Document
Everything

Review & Improve
After Every Deploy

Safe Release Practices

1. **Plan**

Plan every release with clear goals, scope, and success criteria.

2. **Automate**

Automate build, test, and deployment to reduce human error and speed up delivery.

3. **Test Thoroughly**

Perform automated and manual testing across all layers and environments.

4. **Deploy Gradually**

Use progressive delivery (canary, blue-green, or rolling) to minimize risk.

5. **Monitor Continuously**

Monitor health, logs, and metrics in real time to detect issues early.

6. **Have a Rollback Plan**

Always have a tested rollback plan to quickly restore a stable state.

7. **Secure the Release**

Apply security scans, verify compliance, and protect data.

8. **Communicate**

Communicate early and often with all stakeholders.

SAFE RELEASE PRINCIPLES

Quality First
Never compromise on quality and reliability.

Risk Aware
Identify and mitigate risks before release.

User Focused
Deliver value and a great user experience.

Transparency
Be transparent about changes and impact.

Consistent
Follow repeatable processes every time.

Continuous Improvement
Learn from every release and keep improving.

PRE-RELEASE CHECKLIST

- [x] Requirements and scope are approved
- [x] Code reviewed and merged
- [x] All automated tests passed
- [x] Security and quality scans passed
- [x] Documentation and release notes ready
- [x] Rollback plan prepared and tested
- [x] Stakeholders informed and aligned
- [x] Go/No-Go checklist completed

DURING RELEASE CHECKLIST

- [x] Deployment executed with the chosen strategy
- [x] Monitor key metrics and system health
- [x] Validate functionality and user flows
- [x] Watch for errors, alerts, and anomalies
- [x] Communicate status to stakeholders
- [x] Keep rollback option ready
- [x] Confirm success criteria
- [x] Move to next step or rollback if needed

POST-RELEASE CHECKLIST

- [x] Verify system stability and performance
- [x] Confirm data integrity
- [x] Monitor for issues and user feedback
- [x] Update documentation and runbooks
- [x] Close change record
- [x] Share release summary
- [x] Retrospective and lessons learned
- [x] Plan improvements for next release

Goal: Deliver value to users quickly and safely, with confidence and minimal risk.

Result: Reliable software, happy users, and a stronger organization.

TRY IT YOURSELF — EXERCISE 4: FILL RELEASE CHECKLIST TODOS

Reinforces: the Deployment Checklists you just saw, applied to a partially-filled release-checklist-todo.md for Lab 44.

STEPS (notes/lab44-checklist-todos.md)

Step	What To Do
1 — Create the Template	Create release-checklist-todo.md with six blank fields (digest, smoke, correlation, approval, rollback, secrets).
2 — Fill What You Know	Fill in the process fields now; leave digest blanks for real Lab 44 evidence.
3 — Expand/Contract Note	Add one database expand-before-contract reminder line.
4 — Mark Scope	Mark this checklist explicitly as a pre-lab warmup, not the graded release checklist.

release-checklist-todo.md template

```
- Artifact digest: _____
- Staging smoke CUS-1001: _____
- Correlation lab-request-001: _____
- Approval recorded: _____
- Rollback digest ready: _____
- Secrets confirmed out of Git: _____
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-checklist-todos.md -> notes/lab44-checklist-todos.md (workspace: examples/module-44-exercises).

DEPLOYMENT AUTOMATION BEST PRACTICES

Deployment automation reduces human error, accelerates delivery, and ensures consistent, reliable releases across all environments.

AUTOMATED DEPLOYMENT PIPELINE (TYPICAL FLOW)

1. **Commit** — Push to repo.
2. **Build** — Compile code.
3. **Test** — Unit, integration.
4. **Quality Gate** — Scan & enforce.
5. **Deploy** — Dev/QA/stage/prod.
6. **Verify** — Health checks.
7. **Feedback** — Release complete.

CORE PRINCIPLES

- **Consistency & Repeatability —** the same process, tools, and configs every time.
- **Reliability & Security —** build quality and security checks into the pipeline.
- **Transparency & Collaboration —** visibility into pipeline status across teams.

BEST PRACTICES

- Automate end-to-end
- Use Infrastructure as Code
- Small, frequent deployments
- Feature flags
- Versioned & immutable artifacts
- Automate rollback

KEY TAKEAWAY Automate smart. Deliver better. Standardize, automate, validate, monitor, and continuously improve every deployment.

Deployment Automation Best Practices

1 Automate Everything

Automate the entire deployment process including build, test, security, approval, and release.

2 Keep It Repeatable

Ensure deployments are repeatable and consistent across all environments.

3 Version Everything

Version control all artifacts, configurations, scripts, and infrastructure code.

4 Use Infrastructure as Code

Manage infrastructure using code for consistent and auditable environments.

5 Implement Quality Gates

Add automated quality gates for tests, security scans, and approvals before deployment.

6 Deploy Small and Safe

Use progressive delivery strategies (canary, blue-green, rolling) to reduce risk.

7 Monitor and Validate

Continuously monitor deployments and automatically validate health and performance.

8 Plan for Rollback

Automate rollback procedures and test them regularly to ensure readiness.

9 Secure by Default

Integrate security scanning, secrets management, and compliance checks in the pipeline.

10 Continuously Improve

Collect metrics, get feedback, and continuously optimize your deployment process.

AUTOMATED DEPLOYMENT PIPELINE EXAMPLE

1. Code Commit

- Developer commits code
- Trigger pipeline

2. Build

- Compile code
- Build artifacts
- Store in repository

3. Test & Scan

- Run automated tests
- Security & quality scans
- Enforce quality gates

4. Approval

- Automated/manual approval
- Check policies and compliance

5. Deploy

- Deploy to target environment
- Use safe deployment strategy

6. Monitor

- Monitor health and metrics
- Alert on issues
- Validate success

Feedback Loop

Goal: Deliver high-quality software to users faster, safer, and with confidence.

KEY ENABLERS

Cross-functional Collaboration
Dev, QA, Security, and Ops working together.

Standardized Environments
Use consistent, immutable, and reproducible environments.

Observability
Centralize logs, metrics, and traces for full visibility.

Least Privilege Access
Use role-based access and protect sensitive resources.

Documentation and Runbooks
Maintain up-to-date runbooks and process documentation.

Metrics and KPIs
Track deployment frequency, lead time, success rate, MTTR.

TRY IT YOURSELF — EXERCISE 6: PLAN STAGING SMOKE

Reinforces: the automated verify-and-monitor step from Deployment Automation, planned for Northstar CRM's staging fixtures.

STEPS (notes/lab44-staging-smoke-plan.md)

Step	What To Do
1 — List the Cases	Plan checks: read CUS-1001, optional activate path for CUS-1002, readiness probe, correlation header.
2 — Name the Evidence	List screenshot file names to save under notes/screenshots/lab-44/.
3 — Note What's Forbidden	No production data, no real emails, no secret URLs in any saved evidence.
4 — Mark Scope	This is a plan only -- real execution happens later, in the graded Lab 44.

Smoke script sketch

```
for id in CUS-1001 CUS-1002; do
  curl -H "X-Correlation-Id: lab-request-001" \
       "$CRM_BASE_URL/api/customers/$id"
done
curl -H "X-Correlation-Id: lab-request-001" \
     "$CRM_BASE_URL/actuator/health/readiness"
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 6 before continuing: exercises/exercise-06-staging-smoke-plan.md -> notes/lab44-staging-smoke-plan.md (workspace: examples/module-44-exercises).

LAB OVERVIEW -- CONTINUOUS DELIVERY PIPELINE

Lab 44: Continuous Delivery and Environment Promotion -- Northstar Release Path.

BUSINESS SCENARIO

- Leadership freezes a release rule: the artifact that passed staging gates is EXACTLY what production receives -- identified by digest/checksum, never the mutable tag latest.
- Environment configuration and secrets stay outside the immutable artifact. Rollback names a known-good digest plus a verification check.
- You own the release path for Northstar's Customer Management Platform, serving agents who look up Amina (CUS-1001) and Ravi (CUS-1002).

LEARNING OBJECTIVES

- Distinguish continuous delivery (releasable always) from continuous deployment (auto-prod).
- Promote artifacts by digest/checksum rather than rebuilding per environment.
- Separate environment configuration from immutable application bits.
- Define objective, measurable release gates with evidence links.
- Write release and rollback checklists operators can execute under stress.
- Make an explicit go/no-go decision with residual risk owners.

WHAT YOU BUILD

- lab44-crm: docs/release-plan.md, docs/release-checklist.md, docs/rollback-runbook.md, artifact-manifest.json, staging promotion evidence (digest + smoke), and a rehearsed rollback -- built on top of Lab 43's CI artifact identity.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; correlation ID lab-request-001 appears on every smoke check; depends on Lab 43's CI artifact identity.

LAB TASKS AND DELIVERABLES

lab44-crm -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Map the release flow: commit -> build -> registry -> test -> staging -> production, with named approvers.
2	Define immutable identity: semver, commit SHA, JAR checksum / image digest in artifact-manifest.json.
3	Separate configuration: DB URL, Kafka bootstrap, base URLs, and secrets kept outside the artifact.
4	Create promotion gates: tests, scan results, migration review, smoke checks, and approval -- with evidence.
5	Plan database compatibility: expand-before-contract migrations and rollback limits.
6	Rehearse staging release: promote the exact tested digest; run smoke checks for CUS-1001 / CUS-1002.
7	Practice rollback: redeploy the previous known-good digest; verify service and record timing.
8	Complete the release record: release notes, known issues, and a recorded GO / NO-GO decision.
9	Run failure experiments; confirm no secrets are committed; a peer dry-runs the rollback runbook.

Promotion guard pattern

```
kubectl set image deployment/crm-api \
  crm-api="registry.example.com/training/crm-api@${RELEASE_DIGEST}"
kubectl rollout status deployment/crm-api --timeout=180s
curl -fsS -H "X-Correlation-Id: lab-request-001" "$CRM_BASE_URL/api/customers/CUS-1001"
```

EXPECTED DELIVERABLES

- docs/release-plan.md, docs/release-checklist.md, docs/rollback-runbook.md.
- artifact-manifest.json with real digest/checksum identity.
- Staging promotion evidence (digest match + smoke results).
- Controlled failure / NO-GO or rollback rehearsal evidence.
- No secrets or real customer records committed.

RUBRIC (100 MARKS)

- Environment/Structure 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs 10

KEY TAKEAWAY Promoting the tag latest or rebuilding per environment loses core marks; missing rollback verification caps recovery marks -- prove the manifest digest matches what staging and production actually run.

MODULE 44 SUMMARY

In this module, we explored the principles, practices, and automation techniques required to deliver software continuously and reliably from code commit to production.

KEY CONCEPTS COVERED

CD & Release Management

CD principles, the release management lifecycle, and the four environment tiers.

Deployment Strategies

Blue-Green, Canary, Rolling, and Recreate -- and how to choose between them.

Rollback & Recovery

Rollback strategies, disaster recovery basics, and RTO/RPO planning.

Release Governance

Change management, release approvals, and deployment checklists.

Automation

Deployment automation core principles and best practices for reliable releases.

Hands-on Lab

Built lab44-crm: immutable artifact manifest, promotion gates, and a rehearsed rollback.

MODULE FLOW RECAP

1. **Plan & Build**
2. **Test & Package**
3. **Deploy (Strategy)**
4. **Approve & Release**
5. **Monitor & Rollback**

KEY TAKEAWAY Automate everything you can, but keep quality at the core. Shift security left, use approvals and gates for safe releases, monitor continuously, and plan for rollback before you need it.

KNOWLEDGE CHECK (1 OF 3)

Test your understanding of Continuous Delivery's goal, source control, quality gates, and typical pipeline stages.

1. What is the primary goal of Continuous Delivery?

A. Deliver code quickly

B. Deliver code to production automatically and reliably

C. Reduce the number of environments

D. Eliminate manual testing

2. Which tool is commonly used for source code management in a CD pipeline?

A. Maven

B. Jenkins

C. GitHub

D. Docker

3. In a CD pipeline, what is the purpose of the 'Quality Gates' stage?

A. To deploy to production

B. To automatically approve all changes

C. To validate quality via automated checks and approval

D. To store artifacts in a registry

4. Which of the following is NOT a typical stage in a CD pipeline?

A. Code Commit

B. Build

C. Manual Coding

D. Deploy to Production

KEY TAKEAWAY CD keeps software release-ready with a reliable path to production; GitHub manages source code; Quality Gates combine automated checks with manual approval; 'Manual Coding' is not a pipeline stage.

KNOWLEDGE CHECK (2 OF 3)

Four more questions on Infrastructure as Code, artifact repositories, rollback strategy, and containerization.

5. What is the benefit of using Infrastructure as Code (IaC) in deployments?

A. It reduces the number of servers

B. It ensures consistent, repeatable environments

C. It improves code compilation speed

D. It replaces the need for monitoring

6. Artifact repositories (like Nexus or Artifactory) are used to:

A. Store logs

B. Store and manage build artifacts

C. Host production databases

D. Monitor application performance

7. Which of the following best describes a rollback strategy?

A. Rebuilding the application

B. Manually fixing bugs in production

C. Reverting to a previous stable version

D. Deploying to a new environment

8. Which tool is often used for containerization in a CD pipeline?

A. Jenkins

B. Docker

C. Maven

D. Prometheus

KEY TAKEAWAY IaC ensures consistent, repeatable environments; artifact repositories store and manage build artifacts; rollback means reverting to a previous stable version; Docker handles containerization.

KNOWLEDGE CHECK (3 OF 3)

Two final questions on the purpose of monitoring and why approval gates matter in a release process.

9. What is the main purpose of monitoring in a CD environment?

A. To write better code

B. To reduce build time

C. To detect issues early and ensure system health

D. To approve releases

10. Why are approval gates important in a release process?

A. To slow down the pipeline

B. To ensure changes are reviewed and authorized

C. To replace automated tests

D. To increase manual work

KEY TAKEAWAY Monitoring exists to detect issues early and ensure system health; approval gates exist to ensure changes are reviewed and authorized before production -- not to slow delivery down or replace testing.

"Automate. Validate. Release. Repeat."

MODULE 44 COMPLETE

Continuous Delivery Pipeline

You can now design, promote, and govern a Continuous Delivery pipeline -- environments, deployment strategies, rollback, disaster recovery, governance, and automation best practices.